Airboard project report

An AI assisted webcam whiteboard

Submission deadline: 2 November 2026 at 08:00, Asia/Taipei

Airboard is a Python prototype for drawing on a screen with hand movements. A webcam observes the hand, simple gestures choose a tool, and a separate canvas keeps the drawing. The goal is a small idea that another student can try and understand, with optional AI help for interpreting a saved board.

What I made

The program supports drawing, moving without drawing, erasing, and clearing. It can show the canvas over the camera image or on a white background, change ink colors, and save a PNG. A mouse mode provides a way to try the canvas and demonstrate the interface when a webcam is unavailable.

Hand gesture	Action
Index finger extended	Draw a stroke
Index and middle fingers extended	Erase between the fingertips
Open hand	Hover without adding ink
Fist held for one second	Clear the board once

Who it is for

The intended users are students explaining a quick equation or sketch in a study group or classroom. They can experiment with a webcam instead of buying a drawing tablet. This is a prototype for exploration. It has not been evaluated as an accessibility aid or a replacement for precise handwriting tools.

Project repository: https://github.com/Szobo8Nunez9/airboard

The repository contains the source code, software checks, project report, and presentation slides.

How the prototype works

Camera and drawing pipeline

OpenCV captures and mirrors the camera image. MediaPipe Hands supplies 21 hand landmarks. The program converts those landmarks to screen positions and checks which fingers appear extended. The index fingertip becomes the drawing point. Ink stays in a separate image buffer, so displaying or erasing a stroke does not alter the camera frame.

The fingertip position uses an exponential moving average. At alpha 0.5, the new position receives half the weight and the previous smoothed position receives half. Alpha 1.0 follows the raw coordinates. The filter resets when a stroke ends, so a new stroke does not connect to an old position.

Fixed EMA is the default. An optional adaptive mode varies the smoothing gain during motion. The recorded alpha comparison uses fixed EMA.

```
p[t] = alpha * raw[t] + (1 - alpha) * p[t-1]
```

A fist starts a one-second hold timer. Losing the detected hand interrupts the hold. Holding the fist should produce one clear event, and opening the hand prepares the timer for a future hold. The gesture rules assume a reasonably upright, visible hand. Rotation and occlusion remain limitations.

Optional AI interpretation

Pressing A exports a white-background PNG of the board. Separate provider adapters send that same image to Claude and an OpenAI vision model in parallel. Each response carries the provider, model, elapsed time, and success or error state. The interface keeps the two replies separate.

The providers are optional. Real requests require API keys in a local .env file and may use paid API services. Without keys, the canvas still works and the program explains why interpretation is unavailable. Explicit demo mode shows interface examples and says that no model interpreted the board.

The comparison checks explicit final-answer fields rather than numbers found anywhere in an explanation. A match means the displayed final answers match under a simple normalization rule. It does not prove correctness. Unclear or missing replies produce an unassessed comparison.

How I used AI

Antigravity supplied the initial implementation, sample boards, tests, and draft report and slides. ChatGPT independently inspected the source and reproduced edge cases, then helped repair the program and rewrite the supporting material. The work involved reading generated code, running checks, and correcting claims instead of accepting every generated statement.

Claude was part of the intended architecture and review plan. The available project evidence does not independently verify the earlier review actions attributed to Claude. Those actions are not counted as completed contributions in this report.

What I checked

The checks separate software behavior from performance in the real world. A synthetic coordinate test can check a gesture rule, but it cannot measure a webcam under bright or dim lighting. A demo answer can check a panel, but it cannot measure a model's ability to read handwriting.

Check	Recorded evidence and limit
Automated regression suite	80 passed, 0 failed. Tests use controlled inputs and blocked network calls.
Gesture simulation	Generated landmark trials check the classifier. They are labeled synthetic and preserve IDLE or missing detections.
Smoothing simulation	360 generated points across three paths, replayed at alpha 1.0 and 0.5. The jitter metric decreased, while geometric error increased. This does not measure user handwriting.
Webcam field trials	User confirmed the Terminal launch worked on 1 November. Real camera quality and accuracy remain unmeasured. No camera-accuracy percentage is claimed.
Live AI board evaluation	Not measured; no provider requests made. No live-model accuracy or response-time result is claimed.

Problems found in the first draft

The independent review reproduced six errors. Failed provider calls became prepared correct answers in the benchmark. The agreement heuristic accepted shared equation numbers even when final answers differed. The grader accepted a correct number mentioned as a rejected candidate. An IDLE prediction became HOVER and earned credit. The clear timer survived lost tracking. Provider errors produced blank answer panels.

What the repair changes

The revised flow keeps failures as failures and distinguishes live, demo, and unavailable results. It compares explicit final answers, leaves unassessable comparisons unknown, and uses a fixed human rubric for a real 15-board study. Tracking loss resets the gesture state, and each provider panel shows its own failure message. Synthetic test outputs remain clearly labeled.

Six reproduced issues repaired and covered by regression checks.

Reproducing the software checks

```
python -B -m pytest -q -p no:cacheprovider
```

The repository keeps test source and generated evaluation evidence so another reader can inspect what each check did. The former 100% accuracy claims and unverified review-credit entries are not used as evidence in this report.

Limitations and next steps

What remains uncertain

The user confirmed that the Terminal launch worked on 1 November. Gesture comfort, drawing stability, and performance in the presentation room still need a full rehearsal. Lighting, camera distance, hand angle, and occluded fingers can affect the landmarks. The one-second clear gesture also needs a practical check with a person, even when its timer passes a software test. A clean installation and the camera setup still need validation on the presentation computer.

Real AI interpretation has not been measured in this verification run. An API may reject a key or model, time out, misread a stroke, or produce a confident wrong solution. Exact final-answer matching can also mark equivalent mathematical notation as different. The user must review both replies against the board.

Next experiments

First, collect 25 observed trials for each gesture and record the actual lighting and camera distance for every trial. Keep missed detections in the data. Next, replay the same captured fingertip sequence at alpha 1.0 and 0.5 to compare jitter, lag, and corner shape. Finally, submit the same 15 board images to both live providers and retain full replies, model identifiers, errors, and timing. Grade the answers against a fixed rubric and count matching wrong answers separately from matching correct answers.

Notebook and local text recognition

Notebook mode keeps multiple pages with the original ink beside editable text. Apple Vision reads the page locally. Formatting tidies spacing without rewriting the words. The user reviews uncertain recognition, can edit or undo text changes, and exports the notebook as a PDF. A generated printed-text check passed. Real handwriting accuracy remains unmeasured.

Class demonstration

The planned demonstration starts with one short stroke, then hover, erase, and clear. It switches to the white background and saves a board image, then shows a notebook page and its editable text. Mouse mode is available to show the idea if camera access fails. The AI panels are shown as live only when actual requests succeed. Otherwise the presenter uses the labeled demo and explains the remaining work.

Sources and implementation references

MediaPipe Hands legacy solution
https://chuoling.github.io/mediapipe/solutions/hands.html

OpenAI images and vision
https://developers.openai.com/api/docs/guides/images-vision?api-mode=chat

Anthropic vision input guide
https://platform.claude.com/docs/en/build-with-claude/vision

OpenCV image encoding
https://docs.opencv.org/4.x/d4/da8/group__imgcodecs.html